TP N°3 bis : Manipulation avancée des données avec INSERT, UPDATE et DELETE sous MariaDB

Niveau : Licence 2 – Bases de Données

---

1. Objectifs pédagogiques

À l'issue de ce TP, l'étudiant devra être capable de :

- Maîtriser les commandes DML : INSERT, UPDATE, DELETE
- Insérer des données simples et multiples
- Insérer des données à partir d'une autre table
- Mettre à jour des données avec des conditions
- Utiliser des sous-requêtes dans les mises à jour
- Supprimer des données de manière sécurisée
- Comprendre l'impact des contraintes lors des modifications

---

2. Prérequis

L'étudiant doit disposer de :

- La base de données : **gestion_universite**
- Les tables : etudiants, formations, inscriptions
- Une connaissance élémentaire du SQL (SELECT, INSERT, UPDATE, DELETE)

---

3. Contexte du TP

La base de données universitaire est désormais fonctionnelle.

L'administrateur doit :

- Ajouter régulièrement de nouveaux étudiants
- Mettre à jour leurs informations
- Gérer les inscriptions
- Nettoyer les données incorrectes ou obsolètes

Ces opérations sont réalisées à l'aide du langage **DML (Data Manipulation Language)**.

---

4. Mise en route

Sélection de la base

USE gestion_universite;

Vérification des tables

SHOW TABLES;

---

5. Insertion avancée des données (INSERT)

5.1 Insertion simple

Objectif

Ajouter un nouvel étudiant.

INSERT INTO etudiants (nom, prenom, email, age, sexe, nationalite)

VALUES ('Doe', 'Junior', 'junior.doe@mail.com', 21, 'Masculin', 'Togolaise');

Remarque

Respecter l'ordre et le type des champs.

5.2 Insertion multiple

Objectif

Insérer plusieurs étudiants en une seule requête.

INSERT INTO etudiants (nom, prenom, email, age, sexe, nationalite)

VALUES

('Kossi', 'Marc', 'marc.kossi@mail.com', 22, 'Masculin', 'Togolaise'),

('Afi', 'Claire', 'claire.afi@mail.com', 20, 'Féminin', 'Togolaise');

Avantage

Gain de temps et meilleure performance.

5.3 Insertion à partir d'une autre table (INSERT…SELECT)

Création d'une table d'archivage

CREATE TABLE etudiants_archive AS

SELECT * FROM etudiants WHERE 1 = 0;

Objectif

Archiver les étudiants âgés de plus de 25 ans.

INSERT INTO etudiants_archive

SELECT * FROM etudiants

WHERE age > 25;

Utilité

Sauvegarde et historique des données.

---

6. Mise à jour avancée (UPDATE)

6.1 Mise à jour avec condition

Objectif

Uniformiser la nationalité.

```
UPDATE etudiants
SET nationalite = 'TOGO'
WHERE nationalite = 'Togolaise';
```

Conseil

Toujours vérifier avec SELECT avant.

6.2 Mise à jour avec calcul

Objectif

Mettre à jour l'âge des étudiants.

```
UPDATE etudiants
SET age = age + 1;
```

Attention

Cette requête modifie toute la table.

6.3 Mise à jour avec sous-requête

Objectif

Affecter les étudiants à la formation Informatique.

```
UPDATE inscriptions
SET id_formation = (
    SELECT id_formation
    FROM formations
    WHERE libelle = 'Informatique'
```

```
)
WHERE id_etudiant IN (
   SELECT id_etudiant
   FROM etudiants
);
```

Principe

Une requête interne fournit les valeurs à la requête principale.

---

7. Suppression avancée (DELETE)

7.1 Suppression conditionnelle

Objectif

Supprimer les étudiants de plus de 30 ans.

```
DELETE FROM etudiants
WHERE age > 30;
```

7.2 Suppression avec sous-requête

Objectif

Supprimer les inscriptions sans formation valide.

```
DELETE FROM inscriptions
WHERE id_formation NOT IN (
   SELECT id_formation
   FROM formations
);
```

7.3 Suppression sécurisée

Étape 1 : Vérification

```
SELECT * FROM etudiants
WHERE age < 18;
```

Étape 2 : Suppression

```
DELETE FROM etudiants
```

WHERE age < 18;

Objectif pédagogique

Éviter les suppressions accidentelles.

---

8. Utilisation des transactions (Notion de base – L2)

Exemple

START TRANSACTION;

DELETE FROM etudiants WHERE age > 40;

ROLLBACK; -- Annule

-- ou

COMMIT; -- Valide

Permet de sécuriser les opérations sensibles.

---

9. Bonnes pratiques en DML

- Toujours utiliser SELECT avant UPDATE et DELETE
- Sauvegarder avant modification
- Tester sur une copie
- Respecter les contraintes
- Éviter les requêtes sans WHERE

---

10. Exercices d'application (Travail personnel)

Exercice 1

Insérer trois formations différentes.

INSERT INTO formations (libelle, duree, frais)

VALUES

('Informatique', 24, 150000),

('Gestion', 18, 120000),

('Comptabilité', 12, 100000);

Exercice 2

Inscrire deux étudiants à une même formation.

```
INSERT INTO inscriptions (id_etudiant, id_formation, date_inscription)
VALUES
(2, 1, CURRENT_DATE),
(3, 1, CURRENT_DATE);
```

Version avec sous-requête (plus professionnelle)

```
INSERT INTO inscriptions (id_etudiant, id_formation)
VALUES
(
 (SELECT id_etudiant FROM etudiants WHERE email = 'marc.kossi@mail.com'),
 (SELECT id_formation FROM formations WHERE libelle = 'Informatique')
),
(
 (SELECT id_etudiant FROM etudiants WHERE email = 'claire.afi@mail.com'),
 (SELECT id_formation FROM formations WHERE libelle = 'Informatique')
);
```

Exercice 3

Augmenter les frais d'une formation de 10 %.

Correction

```
UPDATE formations
SET frais = frais * 1.10
WHERE libelle = 'Informatique';
```

Vérification

```
SELECT libelle, frais
FROM formations
WHERE libelle = 'Informatique';
```

Explication

- * 1.10 = augmentation de 10 %
- WHERE limite la modification

Exercice 4

Supprimer les inscriptions datant de l'année précédente.

Étape 1 : Vérification

```
SELECT *
FROM inscriptions
WHERE YEAR(date_inscription) = YEAR(CURRENT_DATE) - 1;
```

Étape 2 : Suppression

```
DELETE FROM inscriptions
WHERE YEAR(date_inscription) = YEAR(CURRENT_DATE) - 1;
```

Explication

- YEAR() extrait l'année
- Supprime uniquement les anciennes données

Exercice 5

Expliquer l'intérêt des sous-requêtes dans UPDATE et DELETE.

Correction théorique attendue

Définition

Une **sous-requête** est une requête SQL incluse dans une autre requête.

Exemple :

```
SELECT *
FROM etudiants
WHERE id_etudiant IN (
   SELECT id_etudiant
   FROM inscriptions
);
```

Intérêts principaux

1. Automatisation

Les sous-requêtes permettent d’obtenir automatiquement des valeurs.

Exemple :

```
UPDATE inscriptions
SET id_formation = (
   SELECT id_formation
   FROM formations
   WHERE libelle = 'Informatique'
);
```

➞ Pas besoin de connaître l’ID.

---

2. Sécurité

Réduction des erreurs humaines.

Sans sous-requête :

```
UPDATE inscriptions SET id_formation = 5;
```

Risque d’erreur.

Avec sous-requête :

```
UPDATE inscriptions
SET id_formation = (
   SELECT id_formation FROM formations WHERE libelle='Informatique'
);
```

Plus fiable.

---

3. Précision

Permet des conditions complexes.

Exemple :

```
DELETE FROM etudiants
WHERE id_etudiant NOT IN (
   SELECT id_etudiant
   FROM inscriptions
);
```

➞ Supprime seulement les non-inscrits.

4. Lisibilité

Le code devient plus compréhensible.